

TECHNISCHE
UNIVERSITÄT
WIEN

Debugging, Sockets

191.002 Operating Systems VU
2026W

Luca Di Stefano, Jaroslav Klapalek, Joel Klimont, Felix Resch,
Peter Puschner
2026-11-10

Part I

Introduction to GDB

Features of a debugger

A debugger allows to inspect the execution of a program as it runs

- Pause/resume execution
- Execute step-by-step
- Inspect and manipulate the value of variables

To enable this, the program must be compiled with additional information

- "Debugging symbols"
- Map the point of execution to source code listens
- Map memory locations to variable names, etc.
- In GCC this is done with `gcc -g`
- Typically disables optimizations

Starting GDB[1]

```
1 $ gdb ./myprogram     # Add -tui for a rich text interface
2 (gdb)                 # This is the gdb prompt
```

- Create and attach to a new process: `run [args]`
- Terminate process started with `run`: `kill`
- Attach to an existing `./myprogram` process: `attach <pid>`
- Detach from a process: `detach`
- Get inline help: `help`
- Exit: `quit`

[1] The GNU Project Debugger: `http://www.gnu.org/software/gdb`

Breakpoints

- Lines where GDB shall pause execution
- Before you run, it is a good idea to set some
- Insert a breakpoint:

File and line number	`break example.c:24`
Function entry	`break main`

- Add conditions: `break <...> if i==10`
- List breakpoints (with their number): `info breakpoints`
- Toggle/remove breakpoint: `disable/enable/delete <number>`

Execution Control in GDB

When GDB is paused (e.g., at a breakpoint):

- Show current location: `where` or `bt`
- Single-line step (over functions): `next`
- Single-line step (entering functions): `step`
- Single machine instruction step: `stepi`
- Continue until next breakpoint: `continue`
- Continue until specific location is reached: `until <file:line>`
- Continue until current function returns: `finish`

GDB: Working with Variables

- Print current value of an expression: `print <expr>`
- Change value: `set variable x=0`
- Print on every break: `display <expr>`

Watchpoints

- Pause execution when a variable is read/written
- Powerful but might be slow (limited HW support)
- Setting a watchpoint:

Write	`watch myvar`
Read	`rwatch myvar`
Both (*access*)	`awatch myvar`

GDB: Working with Signals

- <Ctrl-C> → SIGINT → control to debugger
  (will not terminate your program!)
- Customise signal handling:

  `handle <signal> [[no]stop] [[no]ignore] [[no]print] [[no]pass]`
- Show current settings: `info signals`
- Send a signal to program: `signal <signal>`

Debugging: GDB and `fork`

Before `fork` is executed: `set follow-fork-mode [child|parent]`

Example

```
$ gdb -tui ./forktest
(gdb) break main
(gdb) set follow-fork-mode child
(gdb) run
(gdb) next
(gdb) :
(gdb) continue
(gdb) quit
```

Conditional Compilation for Debug Messages

```
1 #ifdef DEBUG
2 #include <stdio.h>
3 #define debug(msg) (void) fputs(msg, stderr)
4 #else
5 #define debug(msg) /* NOP */
6 #endif // DEBUG
7
8 /* ... */
9 debug("I reached this point");
```

- Compile with `-DDEBUG` to enable debug messages (lines 2–3)
- Otherwise, `debug(...)` will do nothing (line 5)

Part II

Sockets

IPC Methods

- Named resources: synchronization between unrelated processes
  - Shared Memory (explicit)
  - Semaphores (explicit)
  - Named Pipes (implicit)
- Anonymous: same, for related processes (e.g., parent-child)
- Signals (implicit, no data)

Sockets:

- Implicit synchronization between unrelated processes
- Possibly on different machines (rely on a network protocol)

Network protocols: Transport layer

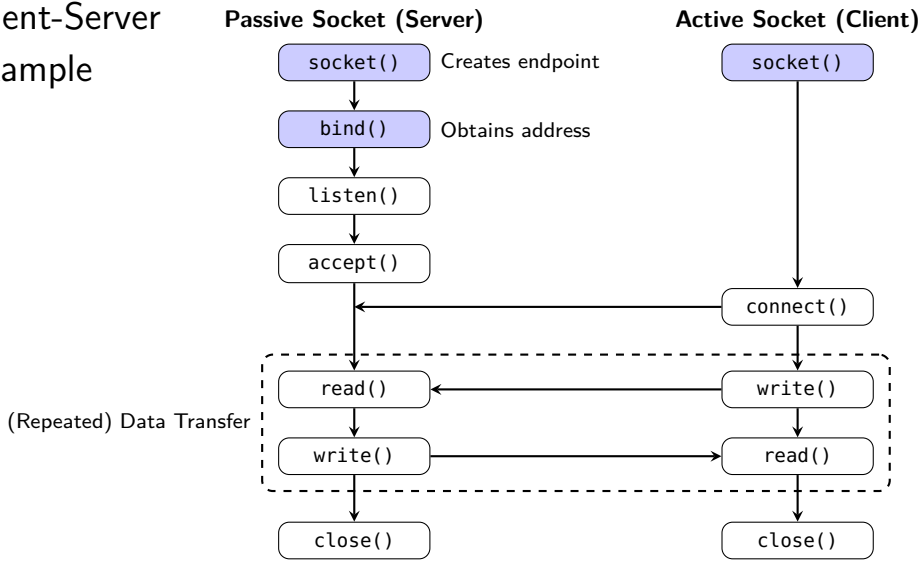
- Networks rely on collections of protocols (e.g., TCP/IP)
- Protocol stack: higher levels build on top of lower ones
- Transport layer: allows processes to communicate across a network
- Sockets allow using transport-layer services, with a file-like interface

TCP: stream-based
- Peers need to establish a connection
- Reliable data transfer

UDP: message-based
- No connection needed
- Messages have fixed size
- Messages might get lost

UNIX domain sockets: local IPC, either stream-based or message-based

Client-Server Example



Creating a socket: `socket(2)`

```c
#include <sys/socket.h>
int socket(int family, int type, int protocol);
```

`family:` "address family"
- `AF_INET` IPv4
- `AF_INET6` IPv6
- `AF_UNIX` UNIX domain

`type:` socket type
- `SOCK_STREAM` connection-oriented
- `SOCK_DGRAM` connection-less

`protocol:` communication protocol, leave 0 for default

`AF_INET` + `SOCK_STREAM` = TCP

`AF_INET` + `SOCK_DGRAM` = UDP

Returns file descriptor of the new socket, or -1 on error ($\rightarrow$ `errno`)

Getting an Address: `bind(2)`

```
int bind(int socket, struct sockaddr *address, socklen_t addr_len);
```

`socket:` socket file descriptor

`address:` pointer to a struct containing the desired address, port, etc.

`addr_len:` size of the address structure

Returns 0 on success, -1 on failure ($\rightarrow$ `errno`)

Typically used by servers to get a well-known address (IP + port)
Clients usually create socket and start using it (will have a random port assigned)

What about `sockaddr`?

Well, it's complicated

- It depends on the address family
  - E.g., for `AF_INET` (IPv4): `struct sockaddr_in`
  - Then, cast to `struct sockaddr*` when needed
- Also, distinction between machine endianness and network byte order (always big endian) for addresses, port numbers, etc.
  - Need to use conversion functions like `htonl`, `ntohl`, ... (`man 3 byteorder`)

But this sounds like a lot of work!

Yes, but thankfully we can use helper functions to set this up

Mainly, `getaddrinfo(3)`

Set up socket data structures: `getaddrinfo(3)`

```
int getaddrinfo(const char *node, const char *service,
                const struct addrinfo *hints,
                struct addrinfo **res);
```

`node`: string with hostname or IP address (or `NULL`)

`service`: port number as string, or name of service like `"http"` (or `NULL`)

`hints`: criteria for selecting socket address structures

`res`: final result will be stored here

Creates a linked list of `addrinfo` structures matching the criteria at `res`

Returns 0 on success, or error code on failure
→ Use `gai_strerror(3)` to get a human-readable error message
→ Use `freeaddrinfo(3)` to free the linked list after use

`getaddrinfo`: Example[2]

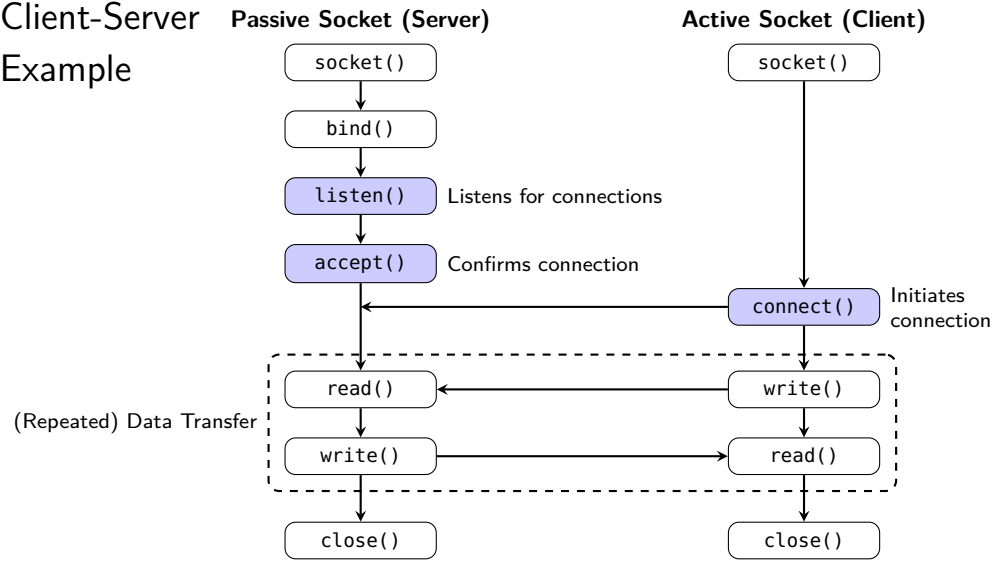
```
1  int status, sockfd;
2  struct addrinfo hints;
3  struct addrinfo *servinfo;
4
5  memset(&hints, 0, sizeof hints); // clear hints structure
6  hints.ai_family = AF_UNSPEC;     // don't care, IPv4 or IPv6
7  hints.ai_socktype = SOCK_STREAM; // TCP
8  hints.ai_flags = AI_PASSIVE;     // Let system figure out the address
9
10 if ((status = getaddrinfo(NULL, "3490", &hints, &servinfo)) != 0) {
11     fprintf(stderr, "Error: %s\n", gai_strerror(status)); exit(EXIT_FAILURE);
12 }
13
14 sockfd = socket(servinfo->ai_family, servinfo->ai_socktype, servinfo->ai_protocol);
15 bind(sockfd, servinfo->ai_addr, servinfo->ai_addrlen);
```

[2] Adapted from https://beej.us/guide/bgnet/html/

Beware of `gethostbyname(3)`

- `getaddrinfo` replaces the obsolete function `gethostbyname`
- E.g., `gethostbyname` does not support IPv6
- Many online examples still use it!
- You **must not** use `gethostbyname` and related functions (i.e. `gethostbyaddr`, `gethostbyname2`, `gethostent_r`, `gethostbyaddr_r`, `gethostbyname_r`, `gethostbyname2_r`, ...) during the exercises nor the exams!

Client-Server Example



Listening for connections: `listen(2)`

(Connection-oriented sockets only)

```
int listen(int socket, int backlog);
```

`socket`: Socket file descriptor

Passive socket: another socket has to initiate the connection

`backlog`: Maximum size of the pending connection queue

A client trying to connect when the queue is full may receive an error (e.g., `ECONNREFUSED`); no effect on the server

Returns (immediately) 0 on success, -1 on failure ($\rightarrow$ `errno`)

Initiating a connection: `connect(2)`

(Connection-oriented sockets only)

```
int connect(int socket, const struct sockaddr *address, socklen_t addr_len);
```

`socket`: socket file descriptor

Active socket: initiates the connection to a server

`address`: address of the server (destination)

`addr_len`: size of the address structure

Blocks until the connection is established or an error occurs

Returns 0 on success, -1 on failure ($\rightarrow$ `errno`)

Confirming a connection: `accept(2)`

(Connection-oriented sockets only)

```
int accept(int socket, struct sockaddr *address, socklen_t *addr_len);
```

`socket`: socket file descriptor

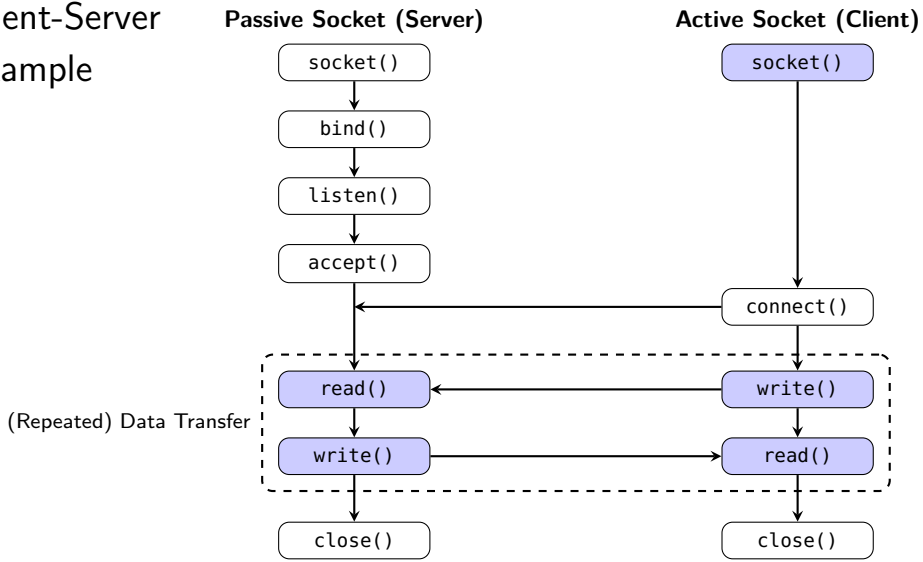
`address`: will be filled with the address of the connecting entity

- Set to `NULL` if not needed
- Again, actual type depends on address family (e.g., `sockaddr_in` for `AF_INET`)

`addr_len`: size of the structure in `address`

Blocks until a connection request arrives
Returns file descriptor of a new socket, or -1 on failure ($\rightarrow$ `errno`)
(The original socket keeps listening, new one is used for communication)

Client-Server Example



Exchanging Data over a Connection

- Once the connection is in place, you can just use `read` and `write`
- Read on empty socket blocks until data arrives
- Write on full socket blocks until space is available
- You can also wrap the socket file descriptor in a `FILE*` (`fdopen(3)`)
- As usual, make sure to flush writes etc.

Stream I/O on Sockets - Example

```c
/* ... (set up hints) */
int res = getaddrinfo("nossl.sh", "http", &hints, &ai);
int sockfd = socket(ai->ai_family, ai->ai_socktype, ai->ai_protocol);
if (sockfd < 0) { /* ... */}
if (connect(sockfd, ai->ai_addr, ai->ai_addrlen) < 0) { /* ... */ }

FILE *sockfile = fdopen(sockfd, "r+");

if (fputs("GET / HTTP/1.1\r\nHost: nossl.sh\r\n\r\n", sockfile) == EOF) { /* ... */ }
if (fflush(sockfile) == EOF) { /* ... */}

char buf[1024];

// print the reply:
while (fgets(buf, sizeof(buf), sockfile) != NULL) fputs(buf, stdout);
```

Send and Receive Data: `send(2)`, `recv(2)`

```
int send(int socket, const void *msg, size_t msg_len, int flags);
int recv(int socket, void* buf, size_t buf_len, int flags);
```

- `write` and `read` operations, specialized for sockets
- Return value and first three arguments: same as `write`/`read`
- Additional flags
  - `MSG_DONTWAIT`: Non-blocking operation (may return -1 with `EAGAIN`/`EWOULDBLOCK` in `errno`)
  - `MSG_WAITALL` (for `recv`): Block until all data was received (or error occurs)

Socket Options: `setsockopt(2)`

```
int setsockopt(int socket, int level, int option_name,
               const void *option_value, socklen_t option_len);
```

Set options on a socket (see man page for full list: setsockopt(2), socket(7), ip(7))

- `socket`: socket file descriptor
- `level`: protocol level (e.g., `SOL_SOCKET` for socket-level options)
- `option_name, option_value, option_len` name of the option to set, pointer to new value, length of the value. Example:

```
1 int optval = 1;
2 setsockopt(serverfd, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof optval);
```

`SO_REUSEADDR` avoids the error "Address already in use" if you try to re-bind to the same port after a crash

Exercise 3

Client and server for HTTP

- 3A: Client, 3B: Server
- IPC via stream-oriented sockets
- Implement a subset of HTTP (HyperText Transfer Procotol), used for requesting websites
- Your server can serve files to a web browser (e.g. Firefox)
- Your client can request files from webservers
  Unfortunately most webservers require HTTPS now. Some exceptions:
  - `http://neverssl.com/`
  - `http://nossl.sh/`

Material

- GDB Tutorial: `http://beej.us/guide/bggdb/`
- GDB Manual: `http://sourceware.org/gdb/onlinedocs/gdb/`
- GDB - Debugging Forks: `https://sourceware.org/gdb/onlinedocs/gdb/Forks.html`
- The GNU C Library Reference Manual,
  Ch. 12 (Stream I/O) and Ch. 16 (Sockets)
  `http://www.gnu.org/software/libc/manual/html_node/`
- Beej's Guide to Network Programming
  `http://beej.us/guide/bgnet/`